Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. doi: 10.18637/jss.v000.i00

CausalMixGPD: Bayesian Conditional Dirichlet Process Mixtures with Generalized Pareto Tails and Causal Extensions

Arnab Aich
Florida State University

Abstract

Heavy-tailed outcomes arise in many scientific domains where rare events dominate risk, cost, or policy decisions. Classical parametric models often fit the center of the distribution at the expense of tail misspecification, while purely nonparametric methods typically lack principled extrapolation in regions with sparse data. Extreme value theory (EVT) provides asymptotic justification for generalized Pareto distributions (GPDs) for threshold exceedances, whereas Bayesian nonparametric Dirichlet process mixtures (DPMs) provide flexible modeling for heterogeneous bulk behavior. This article introduces `CausalMixGPD`, an R package implementing a Bayesian semiparametric framework that combines conditional DPMs for the bulk distribution with covariate-dependent GPD tails for exceedances, with posterior predictive inference for densities, distribution functions, quantiles, survival probabilities, and tail risks. The framework is conditional by construction; unconditional models arise as a special case when covariates are omitted. The package supports both Chinese restaurant process (CRP) and stick-breaking (SB) representations of the Dirichlet process, multiple kernel families for bulk modeling, and an extension to causal inference via treatment-specific conditional distributions that enable inference on distributional and tail-sensitive causal estimands.

Keywords: Bayesian nonparametrics; Dirichlet Process Mixture; Generalized Pareto Distribution; Extreme Value Theory;Causal Inference; R; `nimble` .

1. Introduction

Causal inference is often summarized through average effects, but many applications require distributional comparisons between treatment regimes. In clinical trials, investigators may want to understand how a treatment changes outcomes across the outcome scale, including effects among individuals in the upper tail where events are rare but consequences can be

substantial. This shift from mean-based to distribution-based causal questions makes causal analysis tightly linked to how well we can estimate (and predict) outcome distributions under competing treatment assignments.

A growing R ecosystem supports Bayesian causal inference through flexible outcome modeling and posterior uncertainty quantification. For example, bcf implements Bayesian Causal Forests for binary treatments and continuous outcomes, providing posterior inference for both average and heterogeneous treatment effects (bcf 2024). The package bartCause builds causal estimators around Bayesian additive regression trees, using BART as a regression engine for treatment-response modeling and causal effect estimation (bar 2025; Hill 2011). Complementing tree-based approaches, BCEE implements a Bayesian model-averaging strategy for causal effect estimation with support for binary or continuous exposures and outcomes (BCE 2023; Talbot, Lefebvre, Atherton, and Chiu 2015). A recurring practical difficulty in quantile-based causal reporting is quantile crossing issue, i.e., when two quantile curves for different indices cross each other. To avoid that issue this package directly estimates outcome densities and derives quantiles as functionals of the posterior predictive distribution.

Density estimation and conditional density estimation in R are well supported by Bayesian nonparametric mixture modeling software based on Dirichlet process (DP) ideas. For example, dirichletprocess provides a flexible interface for specifying DP mixture models and fitting them by posterior simulation, yielding posterior inference for mixture weights, component allocations, and predictive densities under user-chosen kernels (dir 2023). The package DPpackage offers a broad suite of Bayesian semi- and nonparametric models in R, including DP-based formulations for marginal and conditional densities, regression functions, and model-based predictive distributions (Jara, Hanson, Quintana, Müller, and Rosner 2011). More recently, BNPmix emphasizes practical Bayesian nonparametric mixture approaches, providing posterior inference for clustering structure and density/regression estimation under DP and Pitman–Yor-type mixture specifications (Corradin, Canale, and Nipoti 2021). However, DP mixture models are suitable for modeling the bulk of the distribution but do not explicitly model tail behavior, which can lead to poor estimation and prediction in the tails when data are sparse or a heavy tail is present.

Bayesian implementations of Extreme Value Theory (EVT) is available in R when posterior uncertainty quantification for tail quantitites are required. For example, evdbayes provides Bayesian inference for standard extreme-value models via posterior simulation, enabling posterior and posterior-predictive summaries for extreme-event quantities (evd 2025). For peaks-over-threshold (POT) approaches, the package POT implements a focused collection of tools for GPD modeling and threshold-based tail analysis (POT 2024). Finally, for extreme quantile estimation and rare-event tail summaries, extremefit provides a published software implementation targeted specifically at extreme-quantile estimation (Durrieu, Grama, Jaunatre, Pham, and Tricot 2018).

This article introduces CausalMixGPD, an R package that connects Bayesian nonparametric bulk modeling with extreme-value tail modeling, while also supporting robust causal analyses. The package provides two complementary capabilities:

- **Bayesian density estimation and prediction with optional GPD tail:** CausalMixGPD fits Dirichlet process mixture models using a user-chosen kernel family for the bulk, and can optionally splice a generalized Pareto tail above a threshold to enable tail-aware estimation and posterior predictive inference for rare-event summaries.

- **Causal estimation and prediction for RCTs and observational studies:** This package also supports causal analyses by modeling treatment-specific potetntial outcome densities and reporting contrasts as functionals of the posterior over densities.

Taken together, CausalMixGPD offers a coherent workflow for density estimation, prediction, and distributional causal reporting when tail behavior is scientifically central.

2. Background and Preliminaries

This chapter treats conditional outcome distributions as the fundamental inferential object. All summaries reported later are functionals of estimated conditional distributions, including conditional quantiles, tail probabilities, and distributional causal contrasts.

Let the outcome–covariate pair be defined by

$$(Y, X) \in \mathbb{R} \times \mathbb{R}^p. \tag{1}$$

For a fixed covariate value x, the conditional cumulative distribution function (CDF) and density are

$$F(y \mid x) = \Pr(Y \leq y \mid X = x), \qquad f(y \mid x) = \frac{\partial}{\partial y} F(y \mid x), \tag{2}$$

and the conditional quantile function is defined by

$$Q(\tau \mid x) = \inf\Big\{y \in \mathbb{R} : \, F(y \mid x) \geq \tau\Big\}, \qquad 0 < \tau < 1. \tag{3}$$

2.1. Dirichlet Process Mixture Model for Conditional Densities

We model the bulk conditional density of Y given $X = x$ using a Dirichlet process mixture (DPM) regression. Let $k(\cdot \mid x; \theta)$ be a user-chosen kernel family indexed by parameter θ. The hierarchical specification is

$$Y \mid X = x, \theta \sim k(\cdot \mid x; \theta), \qquad \theta \mid H \sim H, \qquad H \mid \kappa, H_0 \sim \mathrm{DP}(\kappa, H_0), \tag{4}$$

where H is an unknown mixing distribution on the kernel-parameter space, H_0 is a base measure, and $\kappa > 0$ is a concentration parameter (Ferguson 1973; Lo 1984).

Marginalizing H yields an (almost surely) discrete mixing distribution and an infinite mixture representation:

$$f_{\mathrm{DP}}(y \mid x) = \int k(y \mid x; \theta) \, dH(\theta) = \sum_{j=1}^{\infty} w_j \, k(y \mid x; \theta_j), \qquad w_j \geq 0, \qquad \sum_{j=1}^{\infty} w_j = 1. \tag{5}$$

A constructive representation for the weights is given by Sethuraman's stick-breaking construction (Sethuraman 1994):

$$w_1 = V_1, \qquad w_j = V_j \prod_{\ell < j} (1 - V_\ell), \qquad V_j \overset{\text{iid}}{\sim} \mathrm{Beta}(1, \kappa), \qquad \theta_j \overset{\text{iid}}{\sim} H_0. \tag{6}$$

An alternative marginal representation is obtained by integrating out H, which induces the Blackwell–MacQueen Pólya urn predictive scheme (often described via the Chinese restaurant process) (Blackwell and MacQueen 1973; Neal 2000):

$$\theta_{\text{new}} \mid \theta_{1:n} \sim \frac{\kappa}{\kappa+n} H_0(\cdot) + \frac{1}{\kappa+n} \sum_{m=1}^{n} \delta_{\theta_m}(\cdot). \tag{7}$$

Both stick-breaking (blocked/truncated) and allocation-based representations are widely used for posterior simulation in DPM models (Escobar and West 1995; Neal 2000).

2.2. Generalized Pareto Distribution for Threshold Exceedances

Extreme value theory provides a canonical approximation for exceedances above a high threshold. Let $u(x)$ be a (possibly covariate-dependent) high threshold and define the exceedance

$$Z = Y - u(x), \tag{8}$$

considered conditionally on the event $Y > u(x)$. The conditional excess distribution function is

$$F_u(z \mid x) = \Pr\big(Y - u(x) \le z \mid Y > u(x), X = x\big), \qquad z \ge 0. \tag{9}$$

Under broad regularity conditions, $F_u(\cdot)$ is well-approximated by a generalized Pareto distribution (GPD) for sufficiently large u (Balkema and de Haan 1974; Pickands 1975; Coles 2001).

With scale parameter $\sigma(x) > 0$ and shape parameter $\xi \in \mathbb{R}$, the tail CDF for $Y \mid \{Y > u(x), X = x\}$ is

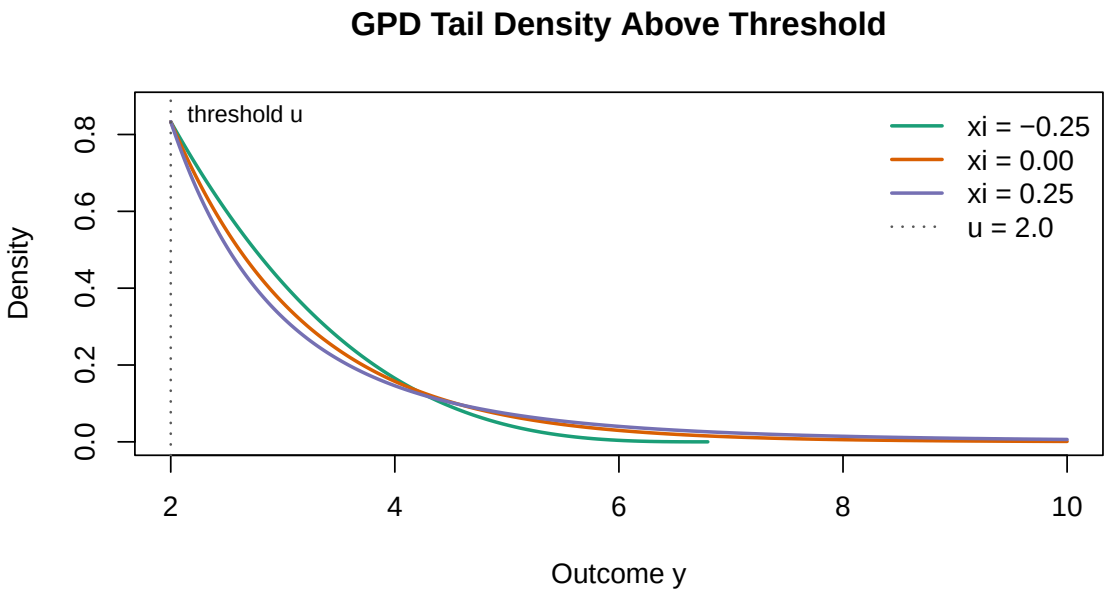
$$F_{\text{GPD}}(y \mid u(x), \sigma(x), \xi, x) = \begin{cases} 1 - \left(1 + \xi \dfrac{y - u(x)}{\sigma(x)}\right)^{-1/\xi}, & \xi \neq 0, \\ 1 - \exp\left(-\dfrac{y - u(x)}{\sigma(x)}\right), & \xi = 0, \end{cases} \qquad y > u(x), \tag{10}$$

with support $y \ge u(x)$ when $\xi \ge 0$, and $u(x) \le y \le u(x) - \sigma(x)/\xi$ when $\xi < 0$. The corresponding density is

$$f_{\text{GPD}}(y \mid u(x), \sigma(x), \xi, x) = \begin{cases} \dfrac{1}{\sigma(x)} \left(1 + \xi \dfrac{y - u(x)}{\sigma(x)}\right)^{-1/\xi - 1}, & \xi \neq 0, \\ \dfrac{1}{\sigma(x)} \exp\left(-\dfrac{y - u(x)}{\sigma(x)}\right), & \xi = 0. \end{cases} \tag{11}$$

The associated GPD quantile function is

$$Q_{\text{GPD}}(\tau \mid u(x), \sigma(x), \xi, x) = \begin{cases} u(x) - \sigma(x) \log(1 - \tau), & \xi = 0, \\ u(x) + \dfrac{\sigma(x)}{\xi}\Big((1 - \tau)^{-\xi} - 1\Big), & \xi \neq 0, \end{cases} \qquad 0 < \tau < 1. \tag{12}$$



2.3. Spliced DPM–GPD Conditional Distribution

Dirichlet process mixture (DPM) regression models provide a highly flexible representation of the *bulk* of the conditional distribution $Y \mid X = x$, accommodating multimodality, skewness, and heterogeneous dispersion through mixture components. However, in the far upper tail—where data are intrinsically sparse—pure mixture-based extrapolation can be unstable and may not reflect the regular variation behavior typically assumed in extreme value theory (EVT). Conversely, EVT provides an asymptotically justified parametric description of threshold exceedances via the generalized Pareto distribution (GPD), but it is not intended to model the full distribution. We therefore adopt a *spliced* (bulk–tail) construction that retains the DPM fit below a high threshold and replaces the upper tail by a GPD component (Frigessi, Haug, and Rue 2002; Behrens, Lopes, and Gamerman 2004; Coles 2001).

To make the splicing and downstream functionals explicit, we collect the unknown bulk regression parameters into Θ and the tail parameters into Φ. Specifically, the bulk model in (5) can be written as

$$f_{\mathrm{DP}}(y \mid x; \Theta) = \sum_{j=1}^{\infty} w_j \, k(y \mid x; \theta_j(x)) \,, \qquad \Theta = \{\beta_j\}_{j \geq 1}, \tag{13}$$

where β_j denotes component-specific regression/shape coefficients and $\theta_j(x)$ denotes the kernel parameter vector induced at covariate value x through the corresponding coefficients in Θ. Mixture weights $\{w_j\}$ are generated by the DP backend (SB or CRP representation) and are suppressed in this compact notation.

For the tail block, let

$$\Phi = \{\beta_u, \beta_\sigma, \xi\}, \tag{14}$$

where $u(x)$ and $\sigma(x)$ are induced by covariates and tail-regression coefficients, and ξ is the tail shape parameter. Thus, Θ and Φ are global parameter collections, while x enters through

these mappings. The package allows users to control which kernel and tail parameters vary with covariates (e.g., via link-based regression for selected parameters) and which are treated as fixed or purely a priori parameters (Section 3).

Let

$$p_u(x;\Theta) = F_{\mathrm{DP}}(u(x) \mid x;\Theta) \tag{15}$$

denote the bulk probability mass below the threshold under the DPM fit. The spliced conditional CDF is

$$F(y \mid x;\Theta,\Phi) = \begin{cases} F_{\mathrm{DP}}(y \mid x;\Theta), & y \le u(x), \\ p_u(x;\Theta) + \{1 - p_u(x;\Theta)\}\, F_{\mathrm{GPD}}(y \mid x;\Phi), & y > u(x), \end{cases} \tag{16}$$

where $F_{\mathrm{GPD}}(\cdot)$ is the GPD CDF on the exceedance region (Section 2.2). The construction is continuous at $u(x)$ and yields a proper distribution: the bulk contributes mass $p_u(x;\Theta)$ and the tail contributes the remaining mass $1 - p_u(x;\Theta)$.

Differentiating (16) yields the spliced density

$$f(y \mid x;\Theta,\Phi) = \begin{cases} f_{\mathrm{DP}}(y \mid x;\Theta), & y \le u(x), \\ \{1 - p_u(x;\Theta)\}\, f_{\mathrm{GPD}}(y \mid x;\Phi), & y > u(x), \end{cases} \tag{17}$$

so the tail density is a *scaled* GPD density, with scaling ensuring normalization of $f(\cdot \mid x;\Theta,\Phi)$.

For $y > u(x)$, the conditional survival function decomposes as

$$\Pr(Y > y \mid X = x;\Theta,\Phi) = \{1 - p_u(x;\Theta)\}\Big[1 - F_{\mathrm{GPD}}(y \mid x;\Phi)\Big]. \tag{18}$$

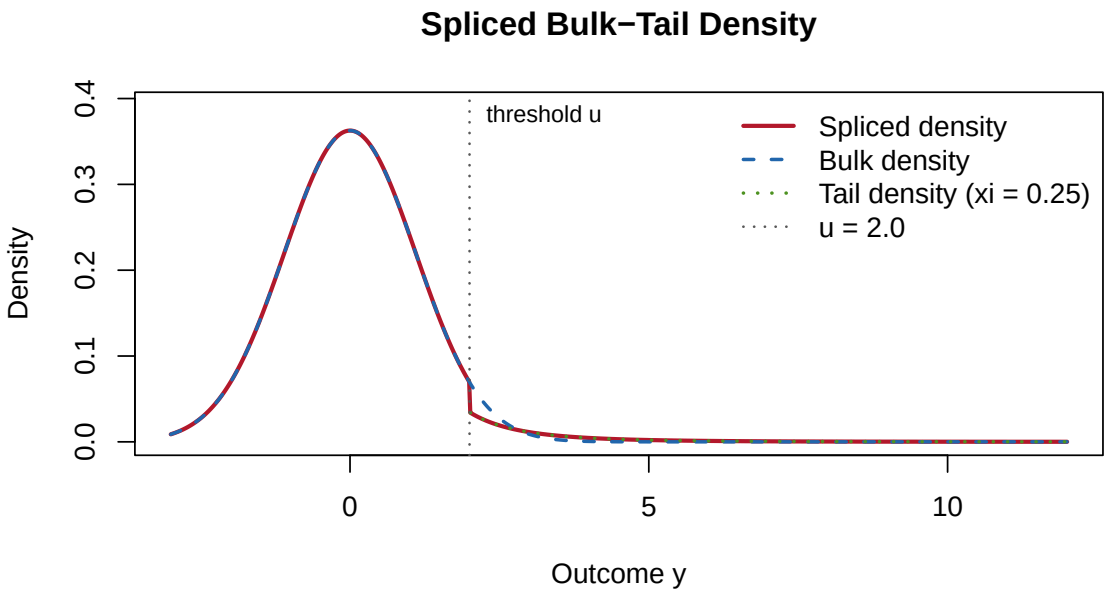
Quantiles follow by inversion of (16). For $\tau \in (0,1)$,

$$Q(\tau \mid x;\Theta,\Phi) = \begin{cases} Q_{\mathrm{DP}}(\tau \mid x;\Theta), & \tau \le p_u(x;\Theta), \\ Q_{\mathrm{GPD}}(\tilde{\tau}(x;\Theta) \mid x;\Phi), & \tau > p_u(x;\Theta), \end{cases} \tag{19}$$

where the tail-rescaled probability level is

$$\tilde{\tau}(x;\Theta) = \frac{\tau - p_u(x;\Theta)}{1 - p_u(x;\Theta)} \in (0,1), \tag{20}$$

and Q_{GPD} is given in (12). Thus, upper quantiles are determined jointly by the bulk exceedance probability $1 - p_u(x;\Theta)$ and the conditional exceedance distribution governed by Φ.



This spliced conditional distribution is the central predictive object used throughout the package. In the causal extension (Section 2.4), the same construction is applied separately to treatment-specific conditional distributions $F_a(\cdot \mid x)$, enabling causal contrasts based on densities, quantiles, and tail risks rather than only mean-scale effects.

2.4. Causal Inference: Notation, Identification, and Reported Estimands

We use the potential outcomes framework (Neyman 1990; Rubin 1974; Imbens and Rubin 2015). Let A denote a binary treatment indicator, let X denote observed pre-treatment covariates, and let Y denote the observed outcome. Each unit has two potential outcomes, $Y(1)$ and $Y(0)$, and the observed outcome satisfies

$$Y = A\,Y(1) + (1 - A)\,Y(0). \tag{21}$$

The treatment-specific conditional distribution functions are the fundamental inferential objects:

$$F_a(y \mid x) = \Pr\{Y(a) \le y \mid X = x\}, \qquad a \in \{0, 1\}. \tag{22}$$

In this package, the corresponding conditional densities are

$$f_a(y \mid x) = \frac{\partial}{\partial y} F_a(y \mid x) \tag{23}$$

where $f_a(y \mid x)$ assumes the spliced DPM structure in (17). Conditional quantiles are defined by inversion of the conditional CDF:

$$Q_a(\tau \mid x) = \inf\{y : F_a(y \mid x) \ge \tau\}, \qquad \tau \in (0, 1). \tag{24}$$

Conditional means are functionals of the conditional density:

$$\mu_a(x) = \mathrm{E}\{Y(a) \mid X = x\} = \int y\, f_a(y \mid x)\, dy. \tag{25}$$

Marginal (standardized) distributions are obtained by averaging conditional distributions over a covariate distribution. The package reports population-standardized and treated-standardized functionals:

$$F_a^{\mathrm{m}}(y) = \mathrm{E}\{F_a(y \mid X)\}, \qquad F_a^{\mathrm{t}}(y) = \mathrm{E}\{F_a(y \mid X) \mid A = 1\}. \tag{26}$$

The corresponding marginal quantiles are

$$Q_a^{\mathrm{m}}(\tau) = \inf\{y : F_a^{\mathrm{m}}(y) \geq \tau\}, \qquad Q_a^{\mathrm{t}}(\tau) = \inf\{y : F_a^{\mathrm{t}}(y) \geq \tau\}. \tag{27}$$

Marginal means are

$$\mu_a^{\mathrm{m}} = \mathrm{E}\{\mu_a(X)\}, \qquad \mu_a^{\mathrm{t}} = \mathrm{E}\{\mu_a(X) \mid A = 1\}. \tag{28}$$

Identification of these quantities depends on the study design. In randomized experiments, randomization together with positivity implies exchangeability between treatment assignment and potential outcomes (Imbens and Rubin 2015). In observational studies, we rely on the following conditions (Rosenbaum and Rubin 1983; Imbens and Rubin 2015):

- **SUTVA:** no interference between units and no hidden versions of treatment.
- **Consistency:** if $A = a$, then the observed outcome equals the corresponding potential outcome, $Y = Y(a)$.
- **Conditional ignorability:** treatment is as-if randomized given covariates,

$$\big(Y(0), Y(1)\big) \perp A \mid X. \tag{29}$$

- **Overlap (positivity):** each covariate profile has positive probability of receiving either treatment,

$$0 < \Pr(A = 1 \mid X = x) < 1 \tag{30}$$

  for covariate values of interest.

Under these conditions, the treatment-specific conditional distributions are identified from the observed data:

$$F_a(y \mid x) = \Pr(Y \leq y \mid A = a, X = x), \qquad a \in \{0, 1\}. \tag{31}$$

In observational settings, we use the propensity score (PS) to support the design stage (Rosenbaum and Rubin 1983). The PS is the treatment assignment probability given covariates:

$$\rho(x) = \Pr(A = 1 \mid X = x). \tag{32}$$

A key property of the true propensity score is that conditioning on it balances covariates across treatment groups:

$$A \perp X \mid \rho(X). \tag{33}$$

With estimated scores this balancing property is approximate in finite samples. Following the two-stage strategy described in the thesis, we first estimate $\rho(x)$ using only (A, X) and

then augment the outcome model covariates with the estimated score (Rosenbaum and Rubin 1983):

$$\widehat{\rho}(x) \approx \rho(x), \qquad r(x) = \left(x^{\mathsf{T}}, \widehat{\rho}(x)\right)^{\mathsf{T}}. \tag{34}$$

The conditional outcome distributions are then modeled separately within each treatment arm as functions of the augmented predictor $r(x)$, and causal contrasts are computed as functionals of the estimated treatment-specific distributions.

The package reports the following causal estimands:

- **Average Treatment Effect (ATE):**

$$\mathrm{ATE} = \mu_1^{\mathrm{m}} - \mu_0^{\mathrm{m}}. \tag{35}$$

- **Average Treatment Effect on the Treated (ATT):**

$$\mathrm{ATT} = \mu_1^{\mathrm{t}} - \mu_0^{\mathrm{t}}. \tag{36}$$

- **Quantile Treatment Effect (QTE):** for any $\tau \in (0, 1)$ (Firpo 2007),

$$\mathrm{QTE}(\tau) = Q_1^{\mathrm{m}}(\tau) - Q_0^{\mathrm{m}}(\tau). \tag{37}$$

- **Quantile Treatment Effect on the Treated (QTT):** for any $\tau \in (0, 1)$,

$$\mathrm{QTT}(\tau) = Q_1^{\mathrm{t}}(\tau) - Q_0^{\mathrm{t}}(\tau). \tag{38}$$

- **Conditional Average Treatment Effect (CATE):**

$$\mathrm{CATE}(x) = \mu_1(x) - \mu_0(x). \tag{39}$$

- **Conditional Quantile Treatment Effect (CQTE):** for any $\tau \in (0, 1)$,

$$\mathrm{CQTE}(\tau \mid x) = Q_1(\tau \mid x) - Q_0(\tau \mid x). \tag{40}$$

This section established the main probabilistic and causal building blocks used in the remainder of the work. We first defined conditional distributional targets and introduced a Dirichlet process mixture model to flexibly estimate bulk conditional outcome densities. We then reviewed generalized Pareto modeling for threshold exceedances and constructed a spliced DPM–GPD conditional distribution that preserves the estimated bulk while enabling principled tail extrapolation. Finally, we introduced the potential outcomes framework and the treatment-effect estimands reported in the package, showing how marginal and conditional effects can be obtained as functionals of treatment-specific conditional distributions (with propensity-score augmentation supporting confounding adjustment in observational studies). Subsequent chapters build on these preliminaries to specify the full hierarchical model, posterior computation, and empirical evaluation of tail-aware distributional causal effects.

3. Package Overview: Workflow, Model, and Inference

This section describes the main workflow provided by CausalMixGPD for one-arm conditional modeling and prediction when a generalized Pareto tail is spliced above a covariate-dependent threshold. The causal extension reuses the same modeling, estimation, and prediction machinery after introducing treatment-specific outcome models and a propensity score component; we defer those additions to the causal section.

3.1. Augmented DPM–GPD model, likelihood, and priors

Let the observed data be

$$\mathcal{D} = \{(y_i, x_i)\}_{i=1}^n,$$

```
# Reload package example data and build a modeling data frame
data("nc_posX100_p3_k2", package = "CausalMixGPD")
dat <- data.frame(y = nc_posX100_p3_k2$y, nc_posX100_p3_k2$X)
```

with $x_i \in \mathbb{R}^p$ denoting the covariate vector (including an intercept if desired). The target conditional distribution $F(\cdot \mid x)$ and its spliced DPM–GPD construction are defined in Section 2, with the spliced CDF and density given in (16)–(17). Posterior computation in CausalMixGPD approximates the bulk DPM by a finite truncation with J mixture components,

$$f_{\mathrm{DP}}(y \mid x; \Theta) \approx \sum_{j=1}^{J} w_j \, k(y \mid x; \theta_j), \qquad j = 1, \ldots, J,$$

where j indexes mixture components, $w_j \geq 0$, and $\sum_{j=1}^{J} w_j = 1$. Kernel parameters can be constant or covariate-dependent through link specifications of the form

$$g(\psi_j(x)) = x^\top \beta_{j,\psi}, \tag{41}$$

where $g(\cdot)$ enforces parameter support (e.g., positive scale parameters).

```
# One-arm spliced DP--GPD fit (with caching to avoid expensive recomputation)
fit_cache_path <- file.path(CACHE_DIR, "overview_fit.rds")

if (USE_CACHE && file.exists(fit_cache_path)) {
  # Load cached fit
  fit <- readRDS(fit_cache_path)
  cat("\n=== Loaded cached fit object ===", "\n")
} else {
  # Fit the model
  cat("\n=== Computing fit (this takes time) ===", "\n")
  fit <- dpmgpd(
    formula    = y ~ x1 + x2 + x3,
    data       = dat,
    backend    = "sb",
    kernel     = "lognormal",
    components = 5,
    mcmc = mcmc_fixed
  )
  # Save to cache
```

```
  cat("\n=== Saving fit to cache ===", "\n")
  saveRDS(fit, fit_cache_path)
}

##
## === Loaded cached fit object ===

# Verify fit object was loaded/created
print(paste("Fit object class:", class(fit)))

## [1] "Fit object class: mixgpd_fit" "Fit object class: list"

print(paste("Fit object size:", object.size(fit), "bytes"))

## [1] "Fit object size: 2507264 bytes"
```

For the spliced model, let $u(x)$ be the threshold, $\sigma(x) > 0$ the tail scale, and $\xi \in \mathbb{R}$ the tail shape (as in Section 2.3). Define $\delta_i = \mathbf{1}\{y_i > u(x_i)\}$. Using the spliced density (17) with the truncated bulk mixture, the likelihood is

$$L(\Theta, \Phi; \mathcal{D}) = \prod_{i=1}^{n} f(y_i \mid x_i; \Theta, \Phi), \tag{42}$$

where Θ collects the truncation-level bulk parameters $\{w_j, \theta_j\}_{j=1}^{J}$, and Φ collects the tail parameters governing $u(x)$, $\sigma(x)$, and ξ. Equivalently, the log-likelihood can be written as

$$\begin{aligned} \log L(\Theta, \Phi; \mathcal{D}) \;=\; & \sum_{i=1}^{n} (1 - \delta_i) \log f_{\mathrm{DP}}(y_i \mid x_i; \Theta) \\ & + \sum_{i=1}^{n} \delta_i \Big[\log\{1 - F_{\mathrm{DP}}(u(x_i) \mid x_i; \Theta)\} + \log f_{\mathrm{GPD}}(y_i \mid x_i; \Phi) \Big], \end{aligned} \tag{43}$$

where f_{GPD} is given in (11) and $F_{\mathrm{DP}}(\cdot \mid x)$ is the bulk mixture CDF induced by the truncated mixture.

Priors follow the decomposition implied by (6) and (7). Under the stick-breaking (SB) backend, the truncation-level weights are generated through Beta stick breaks (Section 2, (6)), which are represented in nimble using the Beta distribution in model code (see `dbeta`). Under the Chinese restaurant process (CRP) backend, the allocation-based representation is implemented using the CRP distribution in nimble (see `dCRP`), matching the predictive representation in (7). Kernel parameters and tail parameters are assigned weakly informative priors, with regression coefficients $\beta_{j,\psi}$ arising through the link representation (41). The same link construction is used for covariate-dependent $u(x)$ and $\sigma(x)$ when those are modeled as functions of x.

3.2. MCMC algorithm, convergence diagnostics, and parameter extraction

Posterior simulation targets $p(\Theta, \Phi \mid \mathcal{D}) \propto L(\Theta, \Phi; \mathcal{D})\, p(\Theta, \Phi)$ using the likelihood (42) and the prior structure implied by the SB or CRP representation (Section 2, (6)–(7)). Given a

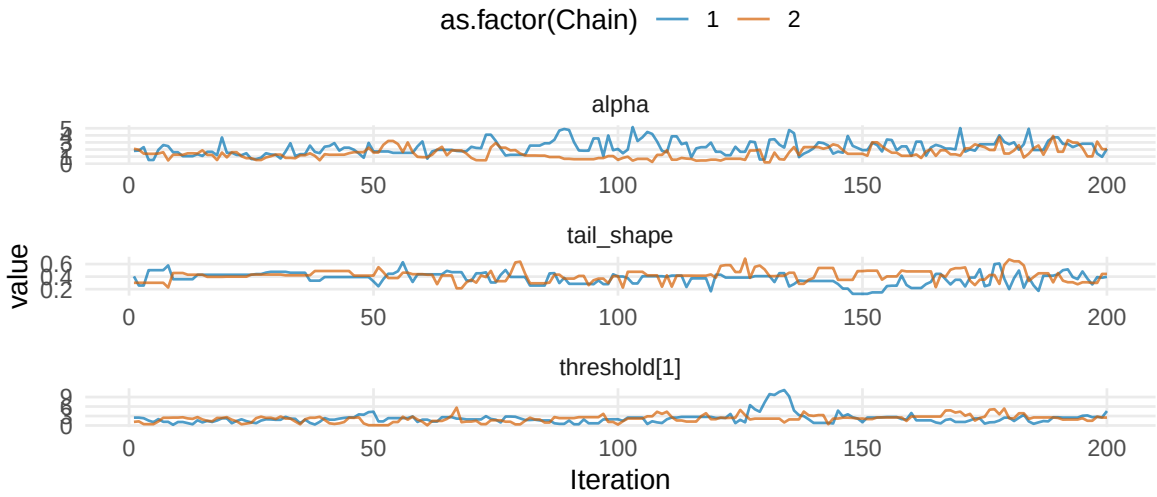
model specification, CausalMixGPD runs MCMC through nimble (de Valpine, Turek, Paciorek, Anderson-Bergman, Temple Lang, and Bodik 2017; nim 2025) under the chosen backend; SB uses Beta stick breaks in model code (see `dbeta`) and CRP uses the CRP distribution for allocations (see `dCRP`). Users control the number of iterations, burn-in, thinning, and number of chains through the `mcmc` argument in the wrappers.

Convergence checking is supported through standard multi-chain diagnostics and trace-based visualizations. The model object supports `summary()` for posterior summaries, `plot()` for diagnostic plots (trace, running mean, autocorrelation, and multi-chain diagnostics such as $\widehat{R}$ when multiple chains are used), and `params()` as a lightweight extractor returning posterior mean parameters reshaped to natural dimensions. Interval summaries used by `predict()` can be chosen as equal-tailed credible intervals or HPD intervals as described in Section 3.3 (Gelman, Carlin, Stern, Dunson, Vehtari, and Rubin 2013; Plummer, Best, Cowles, and Vines 2024). Diagnostic plotting relies on established MCMC diagnostic tooling (ggm 2025).

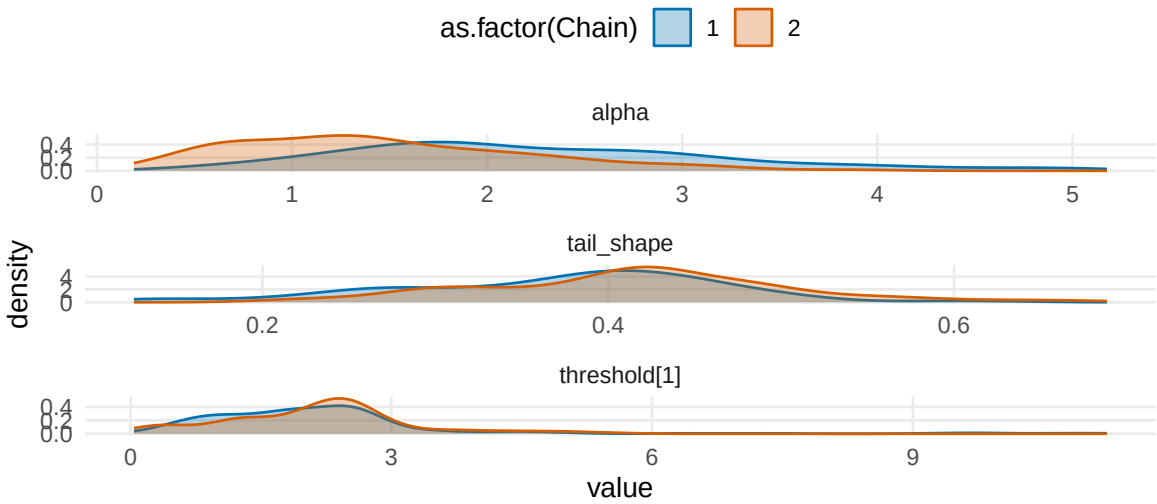
```
# Posterior summaries (includes selected quantiles)
# and may also report ESS where applicable
summary(fit)

## MixGPD summary | backend: Stick-Breaking Process | kernel: Lognormal Distribution | GPD tail: TI
## n = 100 | components = 5
## Summary
## Initial components: 5 | Components after truncation: 3
##
## WAIC: 533.418
## lppd: -190.186 | pWAIC: 76.523
##
## Summary table
##           parameter   mean    sd q0.025 q0.500 q0.975     ess
##          weights[1]   0.46 0.117   0.27   0.45   0.72  11.627
##          weights[2]  0.254 0.061   0.12  0.255   0.36  39.269
##          weights[3]  0.165  0.05   0.08   0.16   0.26  38.227
##               alpha   1.86 0.968  0.487  1.698  4.099  12.727
##  beta_meanlog[1, 1]  0.183 1.477 -2.521 -0.054  3.423   20.74
##  beta_meanlog[2, 1] -0.009 1.399 -3.071 -0.023  2.862  33.912
##  beta_meanlog[3, 1]  0.387  1.74 -3.095  0.301  3.709  43.663
##  beta_meanlog[4, 1]  0.197  1.78 -3.591  0.193  3.467  54.846
##  beta_meanlog[5, 1] -0.334 1.201 -3.053 -0.338  2.421  27.182
##  beta_meanlog[1, 2]  0.047 1.695 -3.234 -0.097   3.69  56.754
##  beta_meanlog[2, 2]  0.028 1.595 -3.422  0.066  3.166  51.988
##  beta_meanlog[3, 2] -0.393 2.104 -4.784 -0.322  3.579  25.438
##  beta_meanlog[4, 2] -0.348 2.033 -4.737 -0.348  3.371   44.29
##  beta_meanlog[5, 2] -0.492 1.505 -2.917 -0.715  3.667  48.203
##  beta_meanlog[1, 3]  0.234 1.263 -2.468  0.197  2.785   42.38
##  beta_meanlog[2, 3] -0.474 1.227 -3.486 -0.377  1.918  25.851
##  beta_meanlog[3, 3]  0.092 1.584 -3.081 -0.089  3.495  18.206
##  beta_meanlog[4, 3]  -0.29 1.817 -3.905 -0.475  3.368  60.839
##  beta_meanlog[5, 3] -0.094 1.216 -2.854 -0.111   2.25  59.107
##  beta_tail_scale[1]  0.377 0.204  0.012  0.357  0.738  30.053
##  beta_tail_scale[2]  0.162 0.302 -0.379  0.137  0.734  51.036
##  beta_tail_scale[3]  0.458 0.153  0.185  0.462  0.769   72.09
##   beta_threshold[1] -0.023 0.147 -0.306 -0.017  0.251  26.686
```

```
##   beta_threshold[2] -0.178 0.234 -0.717 -0.156  0.222   10.31
##   beta_threshold[3] -0.372 0.184 -0.757 -0.366  -0.04   13.39
##             sdlog_u  0.869 0.185  0.489  0.863  1.257    9.34
##          tail_shape  0.393 0.095  0.209  0.406  0.583  49.001
##            sdlog[1]  1.464 0.785  0.351   1.33  3.499  36.647
##            sdlog[2]   2.05 1.297  0.423  1.587  5.209  35.647
##            sdlog[3]  2.041 1.248   0.48  1.791   5.04 101.774

# Extract posterior mean parameters in natural shapes
p_hat <- params(fit)
print(p_hat)

## Posterior mean parameters
##
## $alpha
## [1] "1.86"
##
## $w
## [1] "0.46"  "0.254" "0.165"
##
## $beta_meanlog
##       x1       x2       x3
## comp1 "0.183"  "0.047"  "0.234"
## comp2 "-0.009" "0.028"  "-0.474"
## comp3 "0.387"  "-0.393" "0.092"
## comp4 "0.197"  "-0.348" "-0.29"
## comp5 "-0.334" "-0.492" "-0.094"
##
## $sdlog
## [1] "1.464" "2.05"  "2.041"
##
## $beta_threshold
## [1] "-0.023" "-0.178" "-0.372"
##
## $sdlog_u
## [1] "0.869"
##
## $beta_tail_scale
## [1] "0.377" "0.162" "0.458"
##
## $tail_shape
## [1] "0.393"

# MCMC diagnostics:
# requires ggmcmc; best with multiple chains
plot(fit, family = c("traceplot", "density", "autocorrelation", "Rhat"))

## Scale for colour is already present.
## Adding another scale for colour, which will replace the existing
## scale.
## Scale for colour is already present.
## Adding another scale for colour, which will replace the existing
```

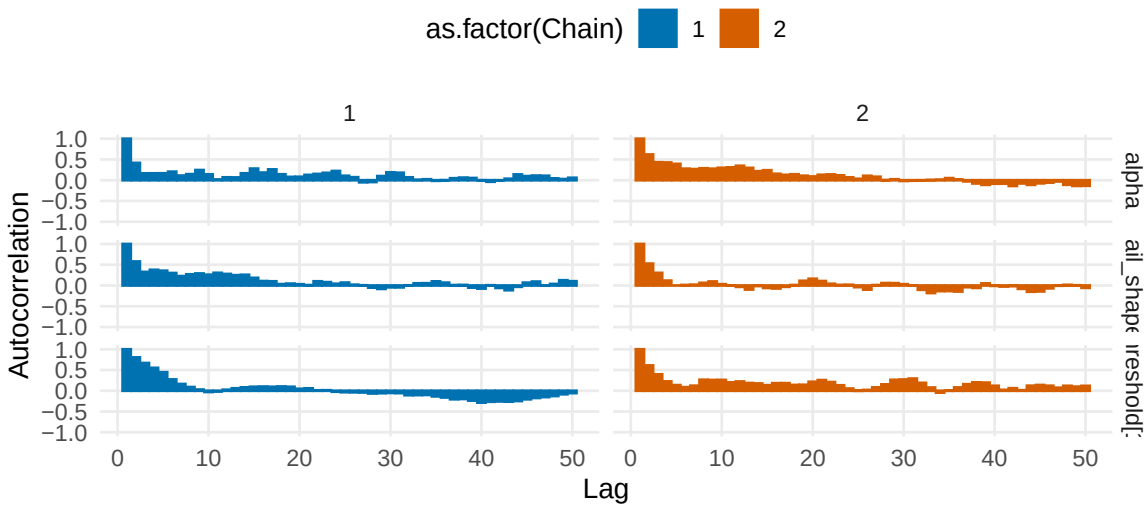
```
## scale.
## Scale for fill is already present.
## Adding another scale for fill, which will replace the existing scale.
## Scale for colour is already present.
## Adding another scale for colour, which will replace the existing
## scale.
## Scale for fill is already present.
## Adding another scale for fill, which will replace the existing scale.
```



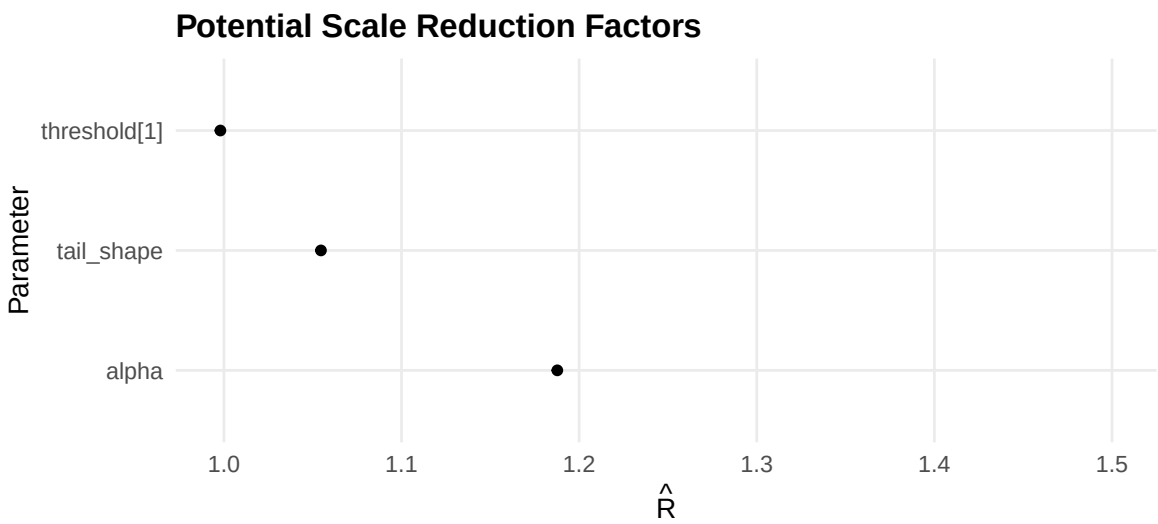

```
##
## === density ===
```



```
##
## === autocorrelation ===
```




```
##
## === Rhat ===
```



Bulk-only special case (no GPD tail)

Setting `GPD=FALSE` removes the splicing step so that $f(y \mid x; \Theta) \equiv f_{\mathrm{DP}}(y \mid x; \Theta)$, and $L(\Theta, \Phi; \mathcal{D})$ reduces to the bulk-mixture likelihood in Θ. All estimation and prediction targets described below remain available, with tail-specific terms omitted. The wrapper `dpmix()` fits this bulk-only model, while `dpmgpd()` fits the augmented (spliced) model.

3.3. Posterior functionals, interval summaries and prediction

The estimands reported by CausalMixGPD are functionals of the estimated conditional distribution $F(\cdot \mid x)$ from Section 2. In this article we focus on four conditional functionals: density $f(y \mid x)$, survival $S(y \mid x) = 1 - F(y \mid x)$, quantiles $Q(\tau \mid x)$, and mean-type summaries.

Let $(\Theta^{(s)}, \Phi^{(s)})_{s=1}^{S}$ denote post-burn-in draws from the posterior. For density, survival, and quantiles, CausalMixGPD evaluates the corresponding draw-wise functional using the spliced formulas (16)–(19) (with the truncated bulk mixture), and then summarizes across draws. Specifically, for a generic scalar functional $T(\Theta, \Phi; x)$ (e.g., $f(y_0 \mid x)$ or $Q(\tau \mid x)$),

$$\widehat{T}(x) = \frac{1}{S} \sum_{s=1}^{S} T(\Theta^{(s)}, \Phi^{(s)}; x),$$

and uncertainty is summarized either by equal-tailed credible intervals (posterior quantiles) or by highest posterior density (HPD) intervals (shortest intervals covering the target posterior mass). Equal-tailed intervals and general Bayesian posterior summaries follow standard practice (Gelman *et al.* 2013); HPD intervals are computed from the MCMC draws using established implementations (Plummer *et al.* 2024).

Mean-type summaries are handled differently in the package. For `type="mean"` and `type="rmean"`, CausalMixGPD uses posterior predictive Monte Carlo within each posterior draw rather than direct plug-in evaluation. For a fixed covariate value x, within draw s the package generates

$$y_1^{(s)}, \ldots, y_M^{(s)} \;\sim\; f(\cdot \mid x; \Theta^{(s)}, \Phi^{(s)}),$$

and computes

$$\mu^{(s)}(x) = \frac{1}{M} \sum_{m=1}^{M} y_m^{(s)}, \qquad \mu_c^{(s)}(x) = \frac{1}{M} \sum_{m=1}^{M} \min\{y_m^{(s)}, c\},$$

corresponding to the conditional mean and the restricted mean at cutoff c, respectively. Posterior summaries and intervals are then formed from $\{\mu^{(s)}(x)\}_{s=1}^{S}$ or $\{\mu_c^{(s)}(x)\}_{s=1}^{S}$. Under the spliced model, the conditional mean is infinite when $\xi \geq 1$; accordingly, CausalMixGPD flags `type="mean"` as infinite when there is posterior mass with $\xi \geq 1$, while `type="rmean"` remains finite by construction.

Using the same fitted model `fit` from Section 3.1, we now compute posterior summaries on the original training covariates (default behavior of `predict()` when `newdata` is not supplied).

```
# Posterior functionals from the same fitted model and data set
q_grid <- c(0.50, 0.90, 0.95, 0.99)
est_quant <- predict(fit, type = "quantile", index = q_grid,
                     interval = "hpd", level = 0.95)
est_mean  <- predict(fit, type = "mean",
                     interval = "credible", level = 0.95)
est_rmean <- predict(fit, type = "rmean", cutoff = 8,
                     interval = "credible", level = 0.95)
# Print a small preview to avoid oversized verbatim output
print(head(est_quant$fit, 10))

##     estimate index id       lower    upper
## 1  1.1517972   0.5  1 0.041727283 2.249710
```

```
## 2  1.0744166   0.5  2 0.043978992 2.267878
## 3  0.9488758   0.5  3 0.001581254 2.168304
## 4  1.1833032   0.5  4 0.166602511 2.330522
## 5  1.1742795   0.5  5 0.103876734 2.225283
## 6  0.9713818   0.5  6 0.002783756 2.086448
## 7  1.0385525   0.5  7 0.002117839 2.174992
## 8  1.3580236   0.5  8 0.069554384 2.659683
## 9  1.1951218   0.5  9 0.118697375 2.412928
## 10 1.2429817   0.5 10 0.046684458 2.365376

print(est_quant$diagnostics)

## $n_draws_total
## [1] 400
##
## $n_draws_valid
## [1] 400
##
## $n_draws_dropped
## [1] 0

print(head(est_mean$fit, 10))

##    id   estimate    lower      upper
## 1   1  18832.394 1.488856  21150.753
## 2   2   1388.221 1.396615   6569.480
## 3   3  31434.621 1.143094   5409.674
## 4   4   6472.091 1.877198   8023.691
## 5   5  10442.449 1.878004   8203.277
## 6   6  10433.525 1.048110   7620.501
## 7   7 226461.009 1.483789 290308.291
## 8   8  61696.016 2.298166   6856.165
## 9   9 424223.534 1.559830   4173.027
## 10 10  32280.834 2.027394  21248.071

print(est_mean$diagnostics)

## $n_draws_total
## [1] 400
##
## $n_draws_valid
## [1] 400
##
## $n_draws_dropped
## [1] 0
##
## $nsim_mean
## [1] 200

print(head(est_rmean$fit, 10))
```

```
##    id estimate     lower    upper
## 1   1 2.595341 1.0869975 3.664537
## 2   2 2.588844 1.0417154 3.934641
## 3   3 2.292212 0.8503162 3.429273
## 4   4 2.775945 1.5829063 3.939630
## 5   5 2.680349 1.5041189 3.689987
## 6   6 2.302220 0.7839485 3.335914
## 7   7 2.513225 1.2177133 3.484890
## 8   8 3.089142 1.5545311 4.490606
## 9   9 2.818617 1.2830895 4.117538
## 10 10 2.776008 1.2800545 4.001016
```

```r
print(est_rmean$diagnostics)
```

```
## $n_draws_total
## [1] 400
##
## $n_draws_valid
## [1] 400
##
## $n_draws_dropped
## [1] 0
##
## $nsim_mean
## [1] 200
```

For a new covariate value $x^\star$, prediction is based on the posterior predictive distribution

$$p(y^\star \mid x^\star, \mathcal{D}) = \int f(y^\star \mid x^\star; \Theta, \Phi)\, p(\Theta, \Phi \mid \mathcal{D})\, d\Theta\, d\Phi.$$

The `predict()` method returns posterior summaries for the same functional targets as in Section 3.3. For `type="density"` and `type="survival"`, the user supplies a grid of y values and the method returns pointwise posterior means with interval bounds computed draw-by-draw. For `type="quantile"`, the user supplies quantile indices τ and the method returns posterior summaries of $Q(\tau \mid x^\star)$ computed using the spliced quantile representation in (19). For `type="mean"` and `type="rmean"`, prediction uses posterior predictive Monte Carlo as described above, with x replaced by $x^\star$ and with user-controlled M through `nsim_mean`.

```r
# Check that fit object exists
if (!exists("fit") || is.null(fit)) {
  stop("fit object not found. The overview-fit-model chunk may not have completed successfully.")
}

# New profiles from coordinate-wise quartiles (25%, 50%, 75%) of covariates
q_levels <- c(0.25, 0.50, 0.75)
x_new <- as.data.frame(lapply(
  dat[, c("x1", "x2", "x3")],
  quantile,
  probs = q_levels,
  na.rm = TRUE
```

```r
))
rownames(x_new) <- c("q25", "q50", "q75")

# Density / survival on a y-grid
y_grid <- seq(0, 10, length.out = 200)
pdens <- predict(fit, newdata = x_new, y = y_grid, type = "density",
                 interval = "credible", level = 0.95)
psurv <- predict(fit, newdata = x_new, y = y_grid, type = "survival",
                 interval = "credible", level = 0.95)

# Quantiles at selected indices
p_grid <- c(0.50, 0.90, 0.95, 0.99)
pquant <- predict(fit, newdata = x_new, type = "quantile", index = p_grid,
                  interval = "hpd", level = 0.95)

# Mean and restricted mean (posterior predictive Monte Carlo)
pmean  <- predict(fit, newdata = x_new, type = "mean",
                  interval = "credible", level = 0.95,
                  nsim_mean = 1000)
prmean <- predict(fit, newdata = x_new, type = "rmean", cutoff = 8,
                  interval = "credible", level = 0.95,
                  nsim_mean = 1000)
# Print small previews to avoid LaTeX dimension errors
preview_fit <- function(obj, n = 5) {
  val <- if (is.list(obj) && "fit" %in% names(obj)) obj$fit else obj
  if (is.matrix(val) || is.data.frame(val)) {
    out <- utils::head(as.data.frame(unclass(val)), n)
    rownames(out) <- NULL
    print(out)
  } else {
    str(val, max.level = 1)
  }
}
cat("\n=== pdens (first 5 rows) ===\n")

##
## === pdens (first 5 rows) ===

preview_fit(pdens, 5)

##   id          y       density         lower         upper
## 1  1 0.00000000 1.000000e-300 1.000000e-300 1.000000e-300
## 2  1 0.05025126  1.480055e+00  6.086270e-02  6.050468e+00
## 3  1 0.10050251  1.031492e+00  1.248750e-01  3.613252e+00
## 4  1 0.15075377  7.467670e-01  1.422720e-01  1.917071e+00
## 5  1 0.20100503  5.817260e-01  1.191826e-01  1.430929e+00

cat("\n=== psurv (first 5 rows) ===\n")

##
## === psurv (first 5 rows) ===
```

```
preview_fit(psurv, 5)

##   id          y  survival     lower     upper
## 1  1 0.00000000 1.0000000 1.0000000 1.0000000
## 2  1 0.05025126 0.8711051 0.4155812 0.9987618
## 3  1 0.10050251 0.8087995 0.3432700 0.9925112
## 4  1 0.15075377 0.7646548 0.3215063 0.9846547
## 5  1 0.20100503 0.7317185 0.3063996 0.9732982

cat("\n=== pquant ===\n")

##
## === pquant ===

preview_fit(pquant, 5)

##   estimate index id       lower    upper
## 1 1.009702   0.5  1 0.007511686 1.949263
## 2 1.008598   0.5  2 0.949744433 1.076012
## 3 1.231136   0.5  3 0.115947359 2.582029
## 4 2.696279   0.9  1 1.846158152 3.588812
## 5 3.342304   0.9  2 2.753517503 3.866009

cat("\n=== pmean ===\n")

##
## === pmean ===

preview_fit(pmean, 5)

##   id    estimate    lower    upper
## 1  1    12710.09 1.826129 20339.15
## 2  2 11285798.34 1.981964 18479.92
## 3  3  1251827.02 1.964290 34094.90

cat("\n=== prmean ===\n")

##
## === prmean ===

preview_fit(prmean, 5)

##   id estimate    lower    upper
## 1  1 2.373860 1.111525 3.286360
## 2  2 2.428256 1.865407 2.986207
## 3  3 3.032066 1.421259 4.419306
```

3.4. Causal extension and propensity score augmentation

This package extends the one-arm conditional mixture regression framework to two-arm causal studies by fitting an outcome model under treatment and under control, optionally augmenting the outcome covariates with a propensity score (PS) summary. Throughout this section we write the model in its default *augmented spliced* form (bulk mixture + GPD tail), using the same conditional distributional building blocks introduced earlier (so we do not restate the spliced density; see (17) and the corresponding conditional functionals defined in the previous sections).

Data, model components, and likelihood factorization

Let

$$\mathcal{D} = \{(y_i, a_i, \boldsymbol{x}_i)\}_{i=1}^n, \qquad a_i \in \{0, 1\},$$

where $a_i = 1$ denotes treatment and $a_i = 0$ denotes control. The causal interface consists of two pieces.

First, when PS augmentation is enabled, a binary regression model is used for the treatment mechanism

$$a_i \mid \boldsymbol{x}_i, \boldsymbol{\gamma} \sim \text{Bernoulli}(e_{\boldsymbol{\gamma}}(\boldsymbol{x}_i)), \tag{44}$$

$$h(e_{\boldsymbol{\gamma}}(\boldsymbol{x}_i)) = \gamma_0 + \boldsymbol{x}_i^\top \boldsymbol{\gamma}, \qquad h \in \{\text{logit}, \text{probit}\}, \tag{45}$$

which is consistent with the definition and balancing property in (32)–(33). The package also supports a `"naive"` option, in which $e(\boldsymbol{x})$ is taken to be constant (no covariate dependence).

Second, the outcome model is fit separately within each arm. Denote by $F_a(\cdot \mid \boldsymbol{r})$ the fitted conditional outcome distribution under arm $a \in \{0, 1\}$, where $\boldsymbol{r}$ is the covariate vector used by the outcome model. With PS augmentation, the package constructs an *augmented* covariate vector

$$\boldsymbol{r}(\boldsymbol{x}) = \Big(1,\ \boldsymbol{x}^\top,\ \psi(\widehat{e}(\boldsymbol{x}))\Big)^\top, \tag{46}$$

where $\widehat{e}(\boldsymbol{x})$ is a posterior summary of the PS (posterior mean or median, controlled by `ps_summary`), and $\psi(\cdot)$ is either $\psi(p) = \text{logit}(p)$ or $\psi(p) = p$ (controlled by `ps_scale`). When PS augmentation is disabled, $\boldsymbol{r}(\boldsymbol{x}) = (1, \boldsymbol{x}^\top)^\top$.

Within each arm, the conditional density $f_a(y \mid \boldsymbol{r})$ follows the same mixture regression + optional GPD tail specification described in the one-arm section (kernel choice, link structure $g(\cdot)$ for covariate-dependent parameters, and either `"sb"` or `"crp"` backend for the DP mixture). Let $\boldsymbol{\Theta}_a$ collect the arm-specific mixture and tail parameters. The arm-wise likelihood contributions are

$$L_a(\boldsymbol{\Theta}_a; \mathcal{D}) = \prod_{i:\, a_i = a} f_a(y_i \mid \boldsymbol{r}(\boldsymbol{x}_i); \boldsymbol{\Theta}_a)\,, \qquad a \in \{0, 1\},$$

with $f_a(\cdot \mid \cdot)$ given by the spliced construction already defined (bulk mixture below threshold and GPD exceedances above threshold). Practically, the implementation uses a two-stage workflow: (i) fit the PS model using (44)–(45), then (ii) compute $\widehat{e}(\boldsymbol{x}_i)$ and build $\boldsymbol{r}(\boldsymbol{x}_i)$ in (46) before fitting the two arm-specific outcome models.

Prior specification (PS then outcome)

For the PS regression coefficients, the default prior is independent Normal,

$$\gamma_0 \sim \mathcal{N}(m_0, s_0^2), \qquad \gamma_k \sim \mathcal{N}(m_0, s_0^2),$$

with defaults controlled by `ps_prior`.

For each arm's outcome model, priors follow the same structure as the one-arm case: a DP mixture prior (with either truncated stick-breaking or CRP allocation, implemented in `nimble` (de Valpine *et al.* 2017; nim 2025)) and kernel-/tail-specific priors on the bulk and GPD parameters. Since the causal fit is simply a pair of one-arm fits (treated and control), all customization options for kernels, links, and tail inclusion apply arm-wise.

MCMC algorithm, diagnostics, and parameter extraction

The causal fit runs MCMC in two stages, each with its own iteration controls.

(i) PS stage: posterior simulation for $\boldsymbol{\gamma}$ in (44)–(45) using `mcmc_ps` settings. The PS posterior draws are summarized to obtain $\widehat{e}(\boldsymbol{x}_i)$, which is optionally transformed via `ps_scale` and appended to the outcome covariates as in (46).

(ii) Outcome stage: arm-specific posterior simulation for $\boldsymbol{\Theta}_0$ and $\boldsymbol{\Theta}_1$ using `mcmc_outcome` settings, with either `"sb"` or `"crp"` backend.

For routine convergence checks, users can inspect trace plots and autocorrelation for monitored parameters and (when running multiple chains) compare between-chain and within-chain mixing. The package stores MCMC samples in the fitted object, and standard diagnostics can be applied after converting samples to `coda` objects (e.g., effective sample size, Geweke diagnostics, and HPD intervals via `coda::HPDinterval`; see the `coda` reference at the end of this message). Posterior parameter summaries can be extracted via the exported `params()` method, which returns arm-specific parameter tables for causal fits.

Estimation and interval summaries for causal contrasts

All reported causal quantities are computed as functionals of the fitted arm-specific conditional distributions. Let $T_a(\boldsymbol{x})$ denote any conditional functional under arm a (density, survival, quantile, or mean/restricted mean), computed from $F_a(\cdot \mid \boldsymbol{r}(\boldsymbol{x}))$ using the same definitions as in the one-arm case. The corresponding conditional causal contrast is

$$\Delta_T(\boldsymbol{x}) = T_1(\boldsymbol{x}) - T_0(\boldsymbol{x}).$$

The package computes $\Delta_T(\boldsymbol{x})$ *draw-by-draw* from posterior samples, then summarizes $\Delta_T(\boldsymbol{x})$ by a posterior mean/median and uncertainty intervals. Equal-tailed credible intervals are formed by posterior quantiles, while HPD intervals are computed using `coda::HPDinterval`.

The main causal estimators provided by the package are:

(a) Conditional effects at covariate profiles. For a user-supplied set of covariate rows $\{\boldsymbol{x}_m^\star\}_{m=1}^M$,

$$\mathrm{CATE}(\boldsymbol{x}_m^\star) = \mu_1(\boldsymbol{x}_m^\star) - \mu_0(\boldsymbol{x}_m^\star), \qquad \mathrm{CQTE}(\tau \mid \boldsymbol{x}_m^\star) = Q_1(\tau \mid \boldsymbol{x}_m^\star) - Q_0(\tau \mid \boldsymbol{x}_m^\star),$$

where $\mu_a(\boldsymbol{x}) = \mathbb{E}(Y \mid A = a, \boldsymbol{X} = \boldsymbol{x})$ and $Q_a(\tau \mid \boldsymbol{x})$ is the conditional quantile function. In heavy-tailed settings, $\mu_a(\boldsymbol{x})$ can be unstable when the implied tail shape yields infinite

mean; the package therefore supports both `type="mean"` and `type="rmean"` for (restricted) mean-based effects, with restricted means computed as $\mathbb{E}[\min\{Y, c\} \mid A = a, \boldsymbol{X} = \boldsymbol{x}]$ at user-specified cutoff c.

(b) Marginal (standardized) summaries. The package reports marginal ATE and marginal QTE-style summaries by empirical standardization over covariate profiles:

$$\mathrm{ATE} = \frac{1}{n}\sum_{i=1}^{n}\Big\{\mu_1(\boldsymbol{x}_i) - \mu_0(\boldsymbol{x}_i)\Big\}, \tag{47}$$

$$\mathrm{QTE}(\tau) = Q_1^{\mathrm{m}}(\tau) - Q_0^{\mathrm{m}}(\tau), \qquad Q_a^{\mathrm{m}}(\tau) = \inf\{y : F_a^{\mathrm{m}}(y) \geq \tau\}, \quad F_a^{\mathrm{m}}(y) = \frac{1}{n}\sum_{i=1}^{n} F_a(y \mid \boldsymbol{x}_i). \tag{48}$$

Treated-standardized analogues (ATT, QTT) are obtained by replacing the full-sample covariate empirical distribution with the treated-subsample empirical distribution $\{i : a_i = 1\}$. In the current implementation, `ate()` and `att()` aggregate draw-wise conditional mean contrasts across rows, while `qte()` and `qtt()` compute draw-wise arm-specific marginal quantiles and then contrast them.

Posterior prediction at new covariate profiles

Prediction in the causal setup proceeds by producing arm-specific predictive summaries at new covariates and then differencing when required. For new covariates $\boldsymbol{x}^\star$, the package first predicts the PS (if enabled), forms $\boldsymbol{r}(\boldsymbol{x}^\star)$ via (46), and then evaluates the desired distributional functional under both arms. For `type = "mean"` and `type = "quantile"`, `predict()` returns treated-minus-control summaries and stores arm-specific predictions as attributes; for `type = "density"`, `"survival"`, and `"prob"`, it returns arm-specific summaries directly.

A recommended way to build an interpretable prediction grid is to use coordinate-wise empirical quartiles. Let $q_{k,p}$ be the pth sample quantile of covariate x_k for $p \in \{0.25, 0.5, 0.75\}$. The grid

$$\mathcal{X}_\star = \prod_{k=1}^{p}\{q_{k,0.25}, q_{k,0.5}, q_{k,0.75}\}$$

yields 3^p representative covariate profiles at which CATE, CQTE, survival contrasts, and other predictive contrasts can be reported.

User-facing workflow (code)

The following example uses the formula interface and the high-level wrapper, fitting the augmented spliced model by default.

```
# Load causal data
data("causal_pos500_p3_k2", package = "CausalMixGPD")
dat <- causal_pos500_p3_k2
df  <- data.frame(y = dat$y, A = dat$A, dat$X)

# Causal fit with caching
cfit_cache_path <- file.path(CACHE_DIR, "causal_fit.rds")
```

```r
if (USE_CACHE && file.exists(cfit_cache_path)) {
  # Load cached fit
  cfit <- readRDS(cfit_cache_path)
  cat("\n=== Loaded cached causal fit object ===", "\n")
} else {
  # Fit the causal model
  cat("\n=== Computing causal fit (this takes time) ===", "\n")
  cfit <- dpmgpd(
    formula = y ~ x1 + x2 + x3,
    data    = df,
    treat   = "A",
    backend     = "sb",
    kernel      = "lognormal",
    components = 5,
    PS          = "logit",
    ps_scale    = "logit",
    ps_summary = "mean",
    mcmc_outcome = mcmc_fixed,
    mcmc_ps      = mcmc_fixed
  )
  # Save to cache
  cat("\n=== Saving causal fit to cache ===", "\n")
  saveRDS(cfit, cfit_cache_path)
}

##
## === Loaded cached causal fit object ===

# Marginal (standardized) effects via aggregation across rows
ate_hat <- ate(cfit, level = 0.90, interval = "credible")
qte_hat <- qte(cfit, probs = c(0.50, 0.90, 0.95), level = 0.90, interval = "hpd")

# Conditional effects at specific covariates
Xnew <- df[1:5, c("x1","x2","x3")]
cate_hat <- cate(cfit, newdata = Xnew, type = "mean",  level = 0.90)
cqte_hat <- cqte(cfit, newdata = Xnew, probs = c(0.50, 0.90), level = 0.90)

# Prediction on a coordinate-wise quartile grid
qs    <- c(0.25, 0.50, 0.75)
Xgrid <- expand.grid(lapply(df[c("x1","x2","x3")], quantile, probs = qs))

pred_q <- predict(cfit, newdata = Xgrid, type = "quantile", index = c(0.50, 0.90))
pred_s <- predict(cfit, newdata = Xgrid, type = "survival", y = rep(4, nrow(Xgrid)))

# Arm-specific predictions are attached as attributes
pred_q_trt <- attr(pred_q, "trt")
pred_q_con <- attr(pred_q, "con")
# Print small previews to avoid LaTeX dimension errors
preview_fit <- function(obj, n = 5) {
  val <- if (is.list(obj) && "fit" %in% names(obj)) obj$fit else obj
  if (is.matrix(val) || is.data.frame(val)) {
    out <- utils::head(as.data.frame(unclass(val)), n)
```

```
    rownames(out) <- NULL
    print(out)
  } else {
    str(val, max.level = 1)
  }
}
cat("\n=== ate_hat ===")

##
## === ate_hat ===

preview_fit(ate_hat, 5)

##  num 0.263

cat("\n=== qte_hat ===")

##
## === qte_hat ===

preview_fit(qte_hat, 5)

##   index   estimate      lower     upper
## 1  0.50 -0.1858070 -0.5684089 0.1984604
## 2  0.90  0.5303081 -0.5522200 1.7325775
## 3  0.95  0.8878052 -1.2839501 2.8650204

cat("\n=== cate_hat (first 5 rows) ===")

##
## === cate_hat (first 5 rows) ===

preview_fit(cate_hat, 5)

##  num [1:5] 0.8614 -0.7081 -0.5234 -0.5698 0.0124

cat("\n=== cqte_hat (first 5 rows) ===")

##
## === cqte_hat (first 5 rows) ===

preview_fit(cqte_hat, 5)

##           V1          V2
## 1  0.6285542  0.79466089
## 2 -0.3758891 -0.88761673
## 3 -0.1818878 -0.79264820
## 4 -0.2230081 -0.82476504
## 5  0.0597947 -0.02784151
```

```r
cat("\n=== pred_q (first 5 rows) ===")

##
## === pred_q (first 5 rows) ===

preview_fit(pred_q, 5)

##   id index        ps  estimate      lower     upper
## 1  1   0.5 0.6050975 0.6016878 -0.9012846 2.4230906
## 2  2   0.5 0.6767695 0.2863477 -0.3100379 1.8837698
## 3  3   0.5 0.7282792 0.1592814 -0.2119625 1.3200645
## 4  4   0.5 0.5610086 0.5958078 -0.7207911 1.6059729
## 5  5   0.5 0.6359172 0.2351063 -0.4938029 0.8800404

cat("\n=== pred_s (first 5 rows) ===")

##
## === pred_s (first 5 rows) ===

preview_fit(pred_s, 5)

##   y        ps trt_estimate    trt_lower  trt_upper con_estimate
## 1 4 0.6050975            1 3.165822e-06 0.19931123            1
## 2 4 0.6767695            1 0.000000e+00 0.08313316            1
## 3 4 0.7282792            1 0.000000e+00 0.04613844            1
## 4 4 0.5610086            1 1.890298e-02 0.29114991            1
## 5 4 0.6359172            1 4.418382e-13 0.05829715            1
##      con_lower   con_upper
## 1 3.893639e-03 0.129019190
## 2 2.637693e-06 0.021225202
## 3 1.548748e-10 0.005122867
## 4 4.116094e-02 0.166960166
## 5 6.677226e-04 0.040991629
```

Bulk-only causal mixtures (no GPD tail)

To run the same two-arm regression without a GPD tail, use the bulk-only wrapper `dpmix()` (or set `GPD=FALSE` at bundle construction). All causal estimand and prediction functions (`ate`, `qte`, `cate`, `cqte`, and `predict`) operate the same way, now interpreting $F_a(\cdot \mid \boldsymbol{x})$ as the bulk mixture distribution.

4. Data analysis I: CRP mixture regression (bulk-only) with model-based clustering

This application demonstrates model-based clustering from a *conditional* Dirichlet process (DP) mixture regression, using the Chinese restaurant process (CRP) backend and the bulk-only likelihood (no GPD tail). Clusters are inferred from posterior draws of the latent membership indicators and summarized using a posterior similarity matrix (PSM), which is invariant to label switching.

4.1. Dataset and objective

We use the `Boston` housing data (MASS), treating each census tract as an observational unit. Let

$$\mathcal{D} = \{(y_i, \boldsymbol{x}_i)\}_{i=1}^n,$$

where y_i is the per-capita crime rate `crim` for tract i and $\boldsymbol{x}_i$ is a vector of tract characteristics. The goal is twofold: (i) flexibly model the conditional distribution of a positive, right-skewed outcome and (ii) identify latent groups of tracts with distinct regression patterns.

4.2. CRP mixture regression model (bulk-only)

We fit a DP mixture of lognormal regressions. Conditional on a latent membership $z_i \in \{1, 2, \ldots\}$, the outcome follows the bulk kernel introduced earlier:

$$y_i \mid z_i = k, \boldsymbol{x}_i \sim \text{Lognormal}(\mu_{ik}, \sigma_k)\,, \tag{49}$$

$$\mu_{ik} = g\Big(\boldsymbol{x}_i^\top \boldsymbol{\beta}_k\Big)\,, \qquad g(\eta) = \eta, \tag{50}$$

where $\boldsymbol{\beta}_k$ are component-specific regression coefficients and $\sigma_k > 0$ is the component-specific log-scale standard deviation. The DP prior induces a random partition through the CRP representation,

$$(z_1, \ldots, z_n) \mid \alpha \sim \text{CRP}(\alpha),$$

with $\alpha > 0$ controlling the expected number of occupied clusters. In CausalMixGPD, the CRP backend represents up to `components=K` clusters in a finite NIMBLE model; the realized number of occupied clusters is then monitored and checked against this upper bound.

4.3. Posterior simulation, convergence checks, and parameter extraction

Posterior simulation is carried out by MCMC. We run two chains and assess convergence using standard diagnostics (trace plots, autocorrelation, effective sample sizes, and $\widehat{R}$ when applicable), along with a mixture-specific diagnostic: the posterior distribution of the number of occupied clusters.

A key practical point for this analysis is that we explicitly request link-mode for `meanlog` so the CRP model is a *mixture regression* rather than a purely unconditional mixture. This is done via `param_specs` in the wrapper call below.

```
library(MASS)
data("Boston", package = "MASS")

dat <- Boston
stopifnot(all(dat$crim > 0))

# Cluster fit with caching
fit_clust_cache_path <- file.path(CACHE_DIR, "app_cluster_fit.rds")

if (USE_CACHE && file.exists(fit_clust_cache_path)) {
  # Load cached fit
  fit_clust <- readRDS(fit_clust_cache_path)
  cat("\n=== Loaded cached cluster fit object ===", "\n")
```

```r
} else {
  # Fit the model
  cat("\n=== Computing cluster fit (this takes time) ===", "\n")
  fit_clust <- dpmix(
    formula = crim ~ lstat + rm + nox,
    data = dat,
    backend = "crp",
    kernel  = "lognormal",
    components = 3,
    mcmc = mcmc_fixed
  )
  # Save to cache
  cat("\n=== Saving cluster fit to cache ===", "\n")
  saveRDS(fit_clust, fit_clust_cache_path)
}

##
## === Loaded cached cluster fit object ===

# High-level summaries
summary(fit_clust)

## MixGPD summary | backend: Chinese Restaurant Process | kernel: Lognormal Distribution | GPD tail
## n = 506 | components = 3
## Summary
## Initial components: 3 | Components after truncation: 2
##
## WAIC: 779.665
## lppd: -259.673 | pWAIC: 130.16
##
## Summary table
##   parameter   mean    sd q0.025 q0.500 q0.975    ess
##  weights[1]  0.565 0.084  0.381  0.585  0.684 11.394
##  weights[2]  0.286 0.039  0.203  0.294  0.348 16.833
##       alpha  0.411 0.239  0.077  0.362  0.974    400
##  meanlog[1] -2.205 0.296 -2.667 -2.245 -1.383 10.751
##  meanlog[2]  1.556  1.15 -2.635  1.993  2.289 29.365
##    sdlog[1]   0.98 0.231  0.622   0.95  1.496 21.156
##    sdlog[2]  1.229 0.416  0.311  1.274  2.014  62.58
```

4.4. Posterior clustering summaries

Clustering is driven by posterior draws of the membership vector $z = (z_1, \ldots, z_n)$. We summarize uncertainty in the partition using the posterior similarity matrix

$$\pi_{ii'} = \Pr(z_i = z_{i'} \mid \mathcal{D}), \qquad \widehat{\pi}_{ii'} = \frac{1}{S}\sum_{s=1}^{S} \mathbb{I}\{z_i^{(s)} = z_{i'}^{(s)}\},$$

where $\{z^{(s)}\}_{s=1}^{S}$ are post-burn-in draws. Because $\widehat{\pi}_{ii'}$ is invariant to label switching, it provides a stable basis for cluster visualization (heatmaps) and for selecting a representative hard

partition.

In practice, we (i) compute $\widehat{\pi}_{ii'}$ from the stored MCMC draws, (ii) select a representative partition from the posterior sample using a least-squares rule against the PSM, and (iii) interpret clusters by comparing outcome and covariate profiles across the resulting groups.

```
# Extract membership draws z[i] from the stored MCMC samples
smp <- fit_clust$mcmc$samples  # a list of chains in coda-compatible form

zmat <- do.call(rbind, lapply(smp, function(ch) {
  M <- as.matrix(ch)
  z_cols <- grep("^z\\[[0-9]+\\]$", colnames(M))
  M[, z_cols, drop = FALSE]
}))

# Mixture-specific diagnostic: number of occupied clusters per draw
n_occupied <- apply(zmat, 1, function(z) length(unique(as.integer(z))))
summary(n_occupied)

##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   2.000   3.000   3.000   2.962   3.000   3.000

max(n_occupied)  # should be <= components (here 5)

## [1] 3

# Posterior similarity matrix (subsample draws for speed)
set.seed(1)
S_keep <- min(400, nrow(zmat))
keep   <- sample(seq_len(nrow(zmat)), S_keep)
Zk     <- zmat[keep, , drop = FALSE]

n <- ncol(Zk)
PSM <- matrix(0, n, n)
for (s in seq_len(nrow(Zk))) {
  z <- as.integer(Zk[s, ])
  PSM <- PSM + (outer(z, z, "==") * 1.0)
}
PSM <- PSM / nrow(Zk)

# Representative partition via least-squares to the PSM
ssq <- numeric(nrow(Zk))
for (s in seq_len(nrow(Zk))) {
  z <- as.integer(Zk[s, ])
  A <- outer(z, z, "==") * 1.0
  ssq[s] <- sum((A - PSM)^2)
}
s_star <- which.min(ssq)
z_hat  <- as.integer(Zk[s_star, ])

# Relabel clusters 1,2,... in order of appearance (cosmetic)
cluster <- match(z_hat, unique(z_hat))
```

```
dat$cluster <- factor(cluster)

table(dat$cluster)

##
##   1   2   3
## 319  33 154
```

4.5. Cluster interpretation and fitted distributional summaries

Given the representative clustering, we compare outcome and covariate profiles across clusters. Because the fitted model is a mixture regression, clusters reflect differences in both the typical outcome level and the component-specific regression behavior.

```
# Outcome summaries by cluster
cluster_outcome <- aggregate(
  crim ~ cluster, data = dat,
  FUN = function(v) c(median = median(v), q25 = quantile(v, 0.25), q75 = quantile(v, 0.75))
)
cluster_outcome <- do.call(data.frame, cluster_outcome)
cluster_outcome

##   cluster crim.median crim.q25.25. crim.q75.75.
## 1       1     0.10574      0.05366     0.220825
## 2       2     0.97617      0.65665     1.425020
## 3       3     7.91590      4.45219    13.481600

# Covariate means by cluster (illustrative subset)
cluster_covars <- aggregate(
  cbind(rm, lstat, nox, ptratio, tax, rad) ~ cluster,
  data = dat,
  FUN = mean
)
cluster_covars

##   cluster       rm     lstat       nox  ptratio      tax       rad
## 1       1 6.425803  9.918621 0.4897389 17.96176 309.2915  4.382445
## 2       2 6.096788 13.980303 0.6265152 17.86667 343.9091  4.878788
## 3       3 6.032468 18.032857 0.6738571 19.60455 626.9805 21.253247

# Fitted conditional summaries at observed X (posterior intervals supported)
fit_med <- fitted(fit_clust, type = "quantile", p = 0.50, level = 0.95, interval = "credible")
fit_hi  <- fitted(fit_clust, type = "quantile", p = 0.90, level = 0.95, interval = "credible")
# Print small previews to avoid LaTeX dimension errors
cat("\n=== fit_med (first 5 rows) ===")

##
## === fit_med (first 5 rows) ===

print(head(fit_med$fit, 5))
```

```
## [1] 0.2181031 0.2181031 0.2181031 0.2181031 0.2181031

cat("\n=== fit_hi (first 5 rows) ===")

##
## === fit_hi (first 5 rows) ===

print(head(fit_hi$fit, 5))

## [1] 12.74664 12.74664 12.74664 12.74664 12.74664

# Attach fitted summaries for downstream plots
dat$fit_q50 <- fit_med$fit
dat$fit_q90 <- fit_hi$fit
```

For prediction at new covariate profiles, we use the same `predict()` interface as in the methods section. In this clustering application, prediction is mainly used to visualize how fitted conditional quantiles vary across covariate patterns and to compare those patterns across the inferred clusters.

```
# Coordinate-wise quartile grid for continuous covariates; fix binary indicator chas
qs <- c(0.25, 0.50, 0.75)
Xgrid <- expand.grid(
  lstat   = quantile(dat$lstat, qs),
  rm      = quantile(dat$rm, qs),
  nox     = quantile(dat$nox, qs),
  ptratio = quantile(dat$ptratio, qs),
  tax     = quantile(dat$tax, qs),
  rad     = quantile(dat$rad, qs)
)
Xgrid$chas <- 0

pred_q <- predict(
  fit_clust,
  newdata = Xgrid,
  type = "quantile",
  index = c(0.50, 0.90, 0.95),
  level = 0.95,
  interval = "credible"
)

## Error:
## !  Column names of 'x' do not match training design matrix.

pred_q

##    id index        ps     estimate       lower     upper
## 1   1   0.5 0.6050975  0.601687787 -0.90128460 2.4230906
## 2   2   0.5 0.6767695  0.286347711 -0.31003785 1.8837698
## 3   3   0.5 0.7282792  0.159281403 -0.21196245 1.3200645
```

```
## 4   4   0.5 0.5610086  0.595807775 -0.72079111 1.6059729
## 5   5   0.5 0.6359172  0.235106303 -0.49380292 0.8800404
## 6   6   0.5 0.6910375  0.126461148 -0.35642010 0.6256969
## 7   7   0.5 0.5044688  0.825903788 -0.84575624 2.0015330
## 8   8   0.5 0.5816165  0.258710322 -1.17259552 1.6710599
## 9   9   0.5 0.6402133  0.039084068 -1.19985321 1.8559604
## 10 10   0.5 0.6092926  0.226781222 -1.02100800 2.2083625
## 11 11   0.5 0.6806716  0.109012510 -0.25566976 1.1742086
## 12 12   0.5 0.7318574  0.067498968 -0.17842333 0.7611048
## 13 13   0.5 0.5653151  0.230643812 -0.57181372 1.3414681
## 14 14   0.5 0.6400243  0.028890252 -0.09786728 0.2190612
## 15 15   0.5 0.6948728  0.020183409 -0.26281808 0.2753406
## 16 16   0.5 0.5088019  0.710313847 -1.17684078 1.7515030
## 17 17   0.5 0.5858868 -0.161791268 -1.26465865 1.4134929
## 18 18   0.5 0.6443024 -0.105503996 -0.86955851 1.1102624
## 19 19   0.5 0.6132415 -0.083332998 -1.32485217 1.6162288
## 20 20   0.5 0.6842068 -0.027303793 -0.67590935 0.6603322
## 21 21   0.5 0.7350121  0.003128245 -0.27192525 0.5019562
## 22 22   0.5 0.5694629 -0.028487435 -1.21992058 1.4436219
## 23 23   0.5 0.6438256 -0.161404442 -0.85155243 0.4923431
## 24 24   0.5 0.6983208 -0.060791284 -0.41825929 0.4054092
## 25 25   0.5 0.5131004  0.442586380 -1.80460103 1.7629533
## 26 26   0.5 0.5899546 -0.430914491 -1.62031867 1.4676720
## 27 27   0.5 0.6480797 -0.242601604 -0.97634419 1.3318021
## 28  1   0.9 0.6050975  0.229664268 -2.64704510 2.6176540
## 29  2   0.9 0.6767695  0.230770589 -1.04977439 2.6935010
## 30  3   0.9 0.7282792  0.221310372 -0.72560701 2.8607803
## 31  4   0.9 0.5610086  0.333093539 -1.63258170 2.2350144
## 32  5   0.9 0.6359172 -0.175240793 -1.49228887 1.8590381
## 33  6   0.9 0.6910375 -0.013820147 -1.23793020 1.8836287
## 34  7   0.9 0.5044688  1.404376286 -0.95564253 3.7694288
## 35  8   0.9 0.5816165 -0.045060629 -1.84370228 2.6837261
## 36  9   0.9 0.6402133 -0.312208492 -2.20696306 2.0194202
## 37 10   0.9 0.6092926 -0.236468166 -2.75415606 2.4390839
## 38 11   0.9 0.6806716 -0.188767610 -1.28173313 1.4734588
## 39 12   0.9 0.7318574 -0.022812759 -0.66086694 1.5745198
## 40 13   0.9 0.5653151 -0.044201338 -1.73550188 2.2094102
## 41 14   0.9 0.6400243 -0.766926056 -1.67510999 0.2948765
## 42 15   0.9 0.6948728 -0.325737283 -1.17956690 0.4869145
## 43 16   0.9 0.5088019  1.566643440 -0.41379523 3.4822190
## 44 17   0.9 0.5858868 -0.591567684 -2.17995285 1.8477972
## 45 18   0.9 0.6443024 -0.780610235 -2.18793187 2.3055206
## 46 19   0.9 0.6132415 -0.621620747 -3.01381924 2.8962529
## 47 20   0.9 0.6842068 -0.525830664 -2.03881174 0.9524041
## 48 21   0.9 0.7350121 -0.195624241 -0.88357675 1.0123263
## 49 22   0.9 0.5694629 -0.216360785 -2.13368807 2.3082978
## 50 23   0.9 0.6438256 -1.045095407 -2.29501934 0.6594895
## 51 24   0.9 0.6983208 -0.543704464 -1.47197337 0.3403696
## 52 25   0.9 0.5131004  1.612128401 -1.18127866 3.7447461
## 53 26   0.9 0.5899546 -0.974995490 -2.75087516 2.7189046
## 54 27   0.9 0.6480797 -0.985348925 -2.43472439 2.3044735
```

References

(2023). *BCEE: The Bayesian Causal Effect Estimation Algorithm*. doi:10.32614/CRAN.package.BCEE. R package on CRAN, URL https://cran.r-project.org/package=BCEE.

(2023). *dirichletprocess: Build Dirichlet Process Objects for Bayesian Modelling*. doi:10.32614/CRAN.package.dirichletprocess. R package on CRAN, URL https://cran.r-project.org/package=dirichletprocess.

(2024). *bcf: Causal Inference using Bayesian Causal Forests*. doi:10.32614/CRAN.package.bcf. R package on CRAN, URL https://cran.r-project.org/package=bcf.

(2024). *POT: Generalized Pareto Distribution and Peaks Over Threshold*. doi:10.32614/CRAN.package.POT. R package on CRAN, URL https://cran.r-project.org/package=POT.

(2025). *bartCause: Causal Inference using Bayesian Additive Regression Trees*. doi:10.32614/CRAN.package.bartCause. R package on CRAN, URL https://cran.r-project.org/package=bartCause.

(2025). *evdbayes: Bayesian Analysis in Extreme Value Theory*. doi:10.32614/CRAN.package.evdbayes. R package on CRAN, URL https://cran.r-project.org/package=evdbayes.

(2025). *ggmcmc: Analysis of MCMC Samples and Bayesian Inference*. doi:10.32614/CRAN.package.ggmcmc. R package on CRAN, URL https://cran.r-project.org/package=ggmcmc.

(2025). *nimble: MCMC, Particle Filtering, and Programmable Hierarchical Modeling*. doi:10.32614/CRAN.package.nimble. R package on CRAN, URL https://cran.r-project.org/package=nimble.

Balkema AA, de Haan L (1974). "Residual Life Time at Great Age." *The Annals of Probability*, **2**(5), 792–804. doi:10.1214/aop/1176996548.

Behrens CN, Lopes HF, Gamerman D (2004). "Bayesian Analysis of Extreme Events with Threshold Estimation." *Statistical Modelling*, **4**(3), 227–244. doi:10.1191/1471082X04st075oa.

Blackwell D, MacQueen JB (1973). "Ferguson Distributions Via Pólya Urn Schemes." *The Annals of Statistics*, **1**(2), 353–355. doi:10.1214/aos/1176342372.

Coles S (2001). *An Introduction to Statistical Modeling of Extreme Values*. Springer, London. doi:10.1007/978-1-4471-3675-0.

Corradin R, Canale A, Nipoti B (2021). "BNPmix: An R Package for Bayesian Nonparametric Modeling via Pitman–Yor Mixtures." *Journal of Statistical Software*, **100**(15), 1–47. doi:10.18637/jss.v100.i15.

de Valpine P, Turek D, Paciorek CJ, Anderson-Bergman C, Temple Lang D, Bodik R (2017). "Programming with Models: Writing Statistical Algorithms for General Model Structures with NIMBLE." *Journal of Statistical Software*, **76**(10), 1–27. doi:10.18637/jss.v076.i10.

Durrieu G, Grama I, Jaunatre K, Pham QK, Tricot JM (2018). "extremefit: A Package for Extreme Quantiles." *Journal of Statistical Software*, **87**(12), 1–20. doi:10.18637/jss.v087.i12.

Escobar MD, West M (1995). "Bayesian Density Estimation and Inference Using Mixtures." *Journal of the American Statistical Association*, **90**(430), 577–588. doi:10.1080/01621459.1995.10476550.

Ferguson TS (1973). "A Bayesian Analysis of Some Nonparametric Problems." *The Annals of Statistics*, **1**(2), 209–230. doi:10.1214/aos/1176342360.

Firpo S (2007). "Efficient Semiparametric Estimation of Quantile Treatment Effects." *Econometrica*, **75**(1), 259–276. doi:10.1111/j.1468-0262.2007.00738.x.

Frigessi A, Haug O, Rue H (2002). "A Dynamic Mixture Model for Unsupervised Tail Estimation without Threshold Selection." *Extremes*, **5**(3), 219–235. doi:10.1023/A:1024072610684.

Gelman A, Carlin JB, Stern HS, Dunson DB, Vehtari A, Rubin DB (2013). *Bayesian Data Analysis*. 3 edition. Chapman and Hall/CRC, Boca Raton, FL. doi:10.1201/b16018.

Hill JL (2011). "Bayesian Nonparametric Modeling for Causal Inference." *Journal of Computational and Graphical Statistics*, **20**(1), 217–240. doi:10.1198/jcgs.2010.08162.

Imbens GW, Rubin DB (2015). *Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction*. Cambridge University Press, Cambridge.

Jara A, Hanson T, Quintana F, Müller P, Rosner G (2011). "DPpackage: Bayesian Semi- and Nonparametric Modeling in R." *Journal of Statistical Software*, **40**(5), 1–30. doi:10.18637/jss.v040.i05.

Lo AY (1984). "On a Class of Bayesian Nonparametric Estimates: I. Density Estimates." *The Annals of Statistics*, **12**(1), 351–357. doi:10.1214/aos/1176346412.

Neal RM (2000). "Markov Chain Sampling Methods for Dirichlet Process Mixture Models." *Journal of Computational and Graphical Statistics*, **9**(2), 249–265. doi:10.1080/10618600.2000.10474879.

Neyman J (1990). "On the Application of Probability Theory to Agricultural Experiments. Essay on Principles. Section 9." *Statistical Science*, **5**(4), 465–480.

Pickands J (1975). "Statistical Inference Using Extreme Order Statistics." *The Annals of Statistics*, **3**(1), 119–131. doi:10.1214/aos/1176343003.

Plummer M, Best N, Cowles K, Vines K (2024). *coda: Output Analysis and Diagnostics for MCMC*. R package version 0.19-4.1, URL https://cran.r-project.org/package=coda.

Rosenbaum PR, Rubin DB (1983). "The Central Role of the Propensity Score in Observational Studies for Causal Effects." *Biometrika*, **70**(1), 41–55. doi:10.1093/biomet/70.1.41.

Rubin DB (1974). "Estimating Causal Effects of Treatments in Randomized and Nonrandomized Studies." *Journal of Educational Psychology*, **66**(5), 688–701. doi:10.1037/h0037350.

Sethuraman J (1994). "A Constructive Definition of Dirichlet Priors." *Statistica Sinica*, **4**(2), 639–650.

Talbot D, Lefebvre G, Atherton J, Chiu Y (2015). "The Bayesian Causal Effect Estimation Algorithm." *Journal of Causal Inference*, **3**(2), 207–236. doi:10.1515/jci-2014-0035.

Affiliation:

Firstname Lastname
Affiliation
Address, Country
E-mail: name@address
URL: http://link/to/webpage/

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd